

Graphes Multiplexes

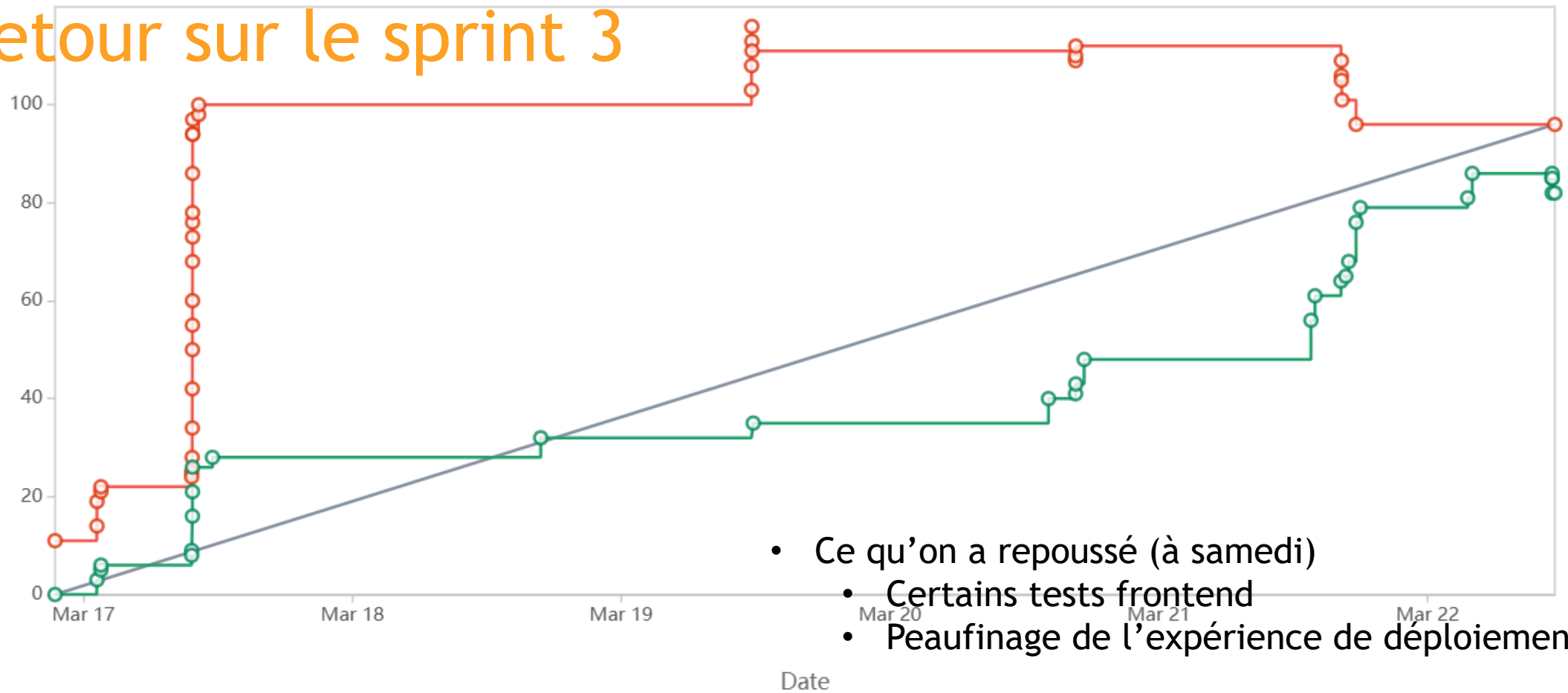
Retour sur la démo du 19/03

- ▶ Difficultés de démarrage (compilation) à cause de la connexion
- ▶ Succès de la démo en elle-même
- ▶ Application mise dans un vrai contexte bio-info pour la première fois

Besoins apparus pendant la démo

- ▶ Documenter les modifications de l'algorithme
- ▶ Ajustements d'affichage (graphe, UI)
 - ▶ Afficher même les graines déconnectées
 - ▶ Boutons verts plus lisibles
 - ▶ Alignement incorrect sous Safari
- ▶ Bug sur les ZIPs

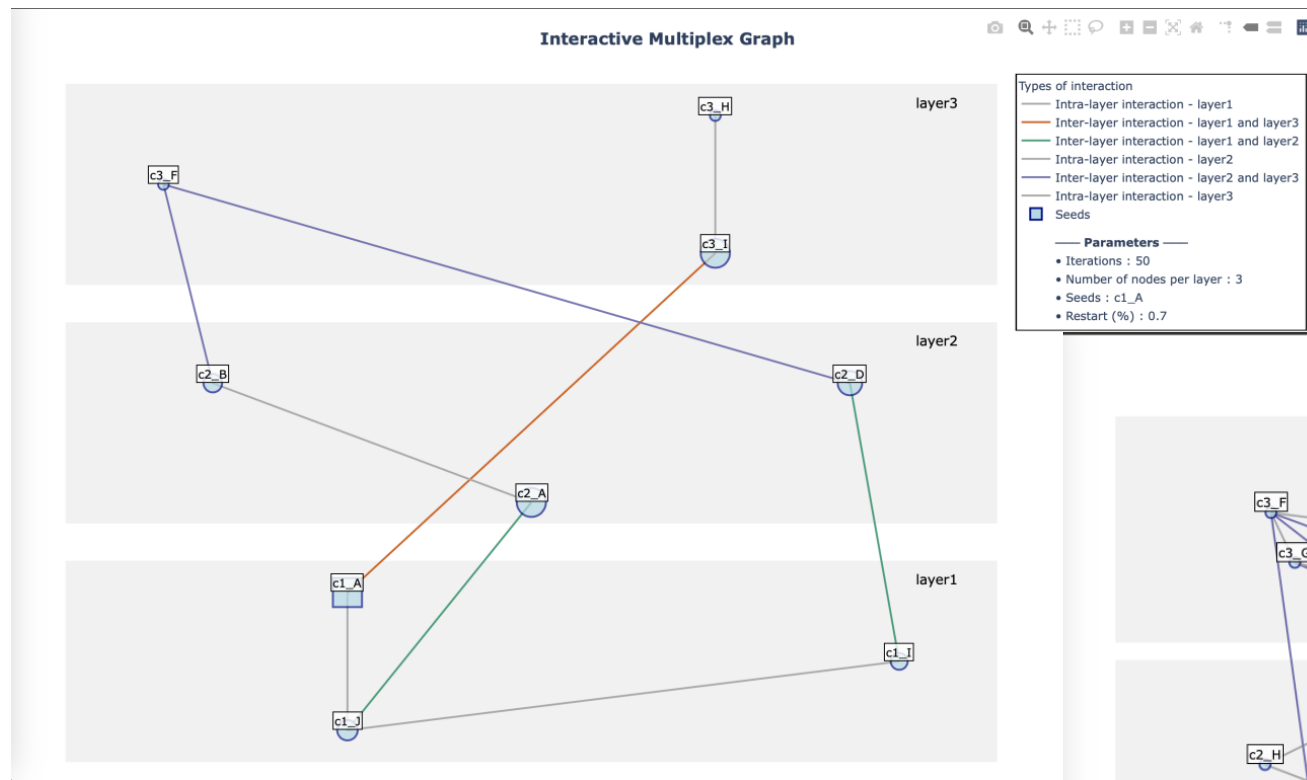
Retour sur le sprint 3



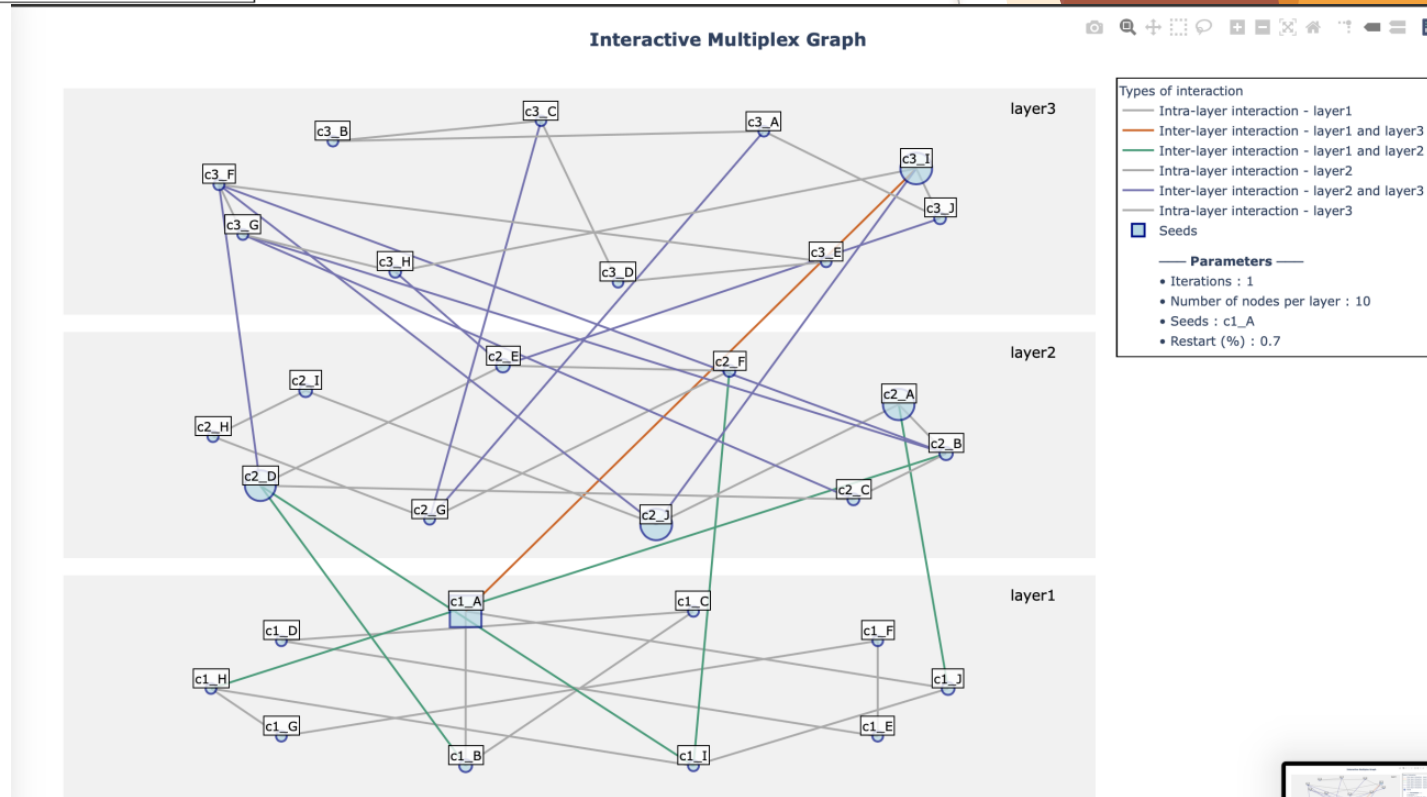
- Ce qu'on a repoussé (à samedi)
 - Certains tests frontend
 - Peaufinage de l'expérience de déploiement

Focus
Frontend

Rappel du sujet



Graphes triviaux



Importation d'un graphe

- ZIP
- Arborescence prédéfinie
- Assistant de création ZIP



Multiplex uploading

(.zip only)

File deposit

[Generate Zip](#)

>>>

Generate Zip

Zip Name:

Enter the zip name

Multiplex Layer

Add Layer

Bipartite

Add Bipartite

demo_itwr_dataset

bipartite

1_2.tsv

1_3.tsv

1_4.tsv

1_5.tsv

multiplex

1

2

HPO_similarity.tsv

3

reactome_similarity.tsv

4

corum_similarity.tsv

5

hepatoDisease_similarity.tsv

layer_name.tsv

Paramétrage

- Sélection d'une couche
- Sélection d'une graine
- Paramètres d'exécution
- UI épurée

The screenshot displays the 'Multiplex' application interface. At the top, there are navigation tabs: 'Multiplex' (active), 'Logs', and 'Credits'. The main content area is divided into two panels. The left panel, titled 'Random walk Algorithm', contains configuration fields: 'Title' (s with 0.7 restart), 'User' (chris), 'Steps' (1), 'Top' (10), 'Restarts' (0,7), and 'Seeds' (acquired immunc). A 'Start' button is at the bottom. The right panel, titled 'Seeds available in the diseases layer', features a search bar and a grid of disease seeds. A 'Layer selection' dropdown menu is open, showing options: Layer, Compounds, Diseases, Genes, and Pathways. The 'Diseases' layer is selected, and the grid shows various diseases, with 'acquired immunodeficiency syndrome' and 'attention deficit hyperactivity disorder' highlighted.

Layer selection

Layer

Layer

Compounds

Diseases

Genes

Pathways

Random walk Algorithm

Title

s with 0.7 restart

User

chris

Steps

1

Top

10

Restarts

0,7

Seeds

acquired immunc

Start

Seeds available in the diseases layer

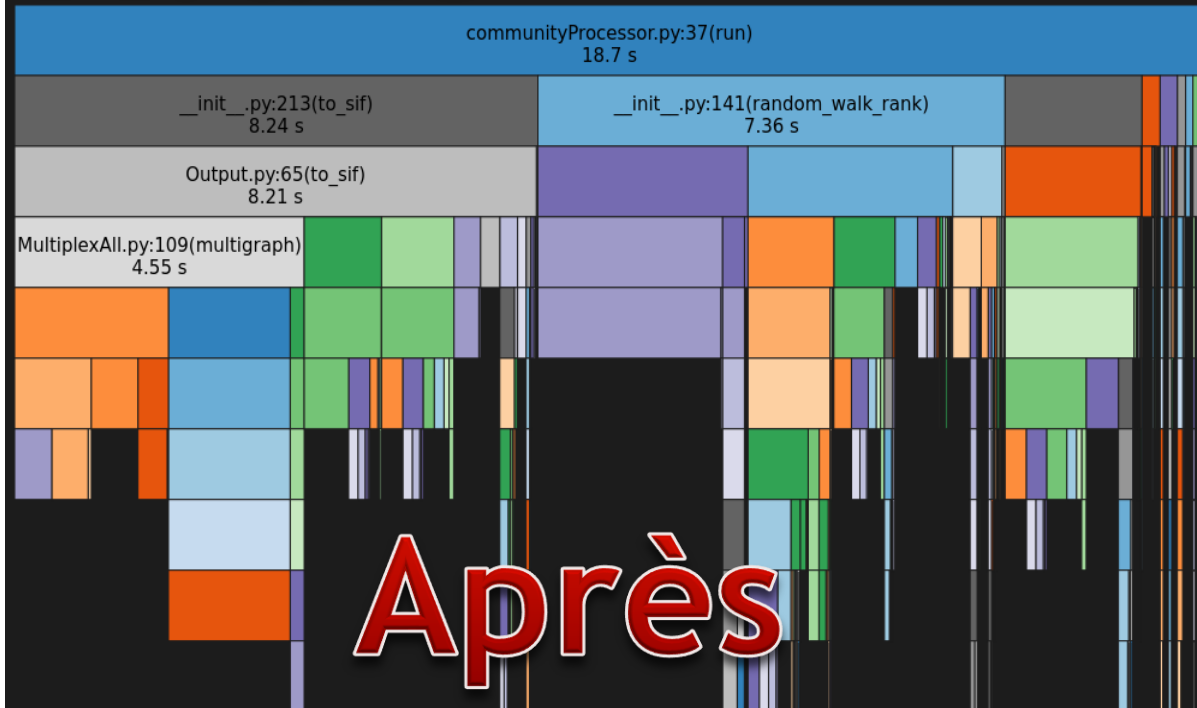
Search seeds...

abdominal aortic aneurysm	acquired immunodeficiency syndrome	adrenal gland cancer	age related macular degeneration	alcohol dependence	allergic rhinitis
alopecia areata	Alzheimer's disease	amyotrophic lateral sclerosis	anemia	ankylosing spondylitis	appendix cancer
asthma	atherosclerosis	atopic dermatitis	attention deficit hyperactivity disorder	autistic disorder	azoospermia
Barrett's esophagus	Behcet's disease	bile duct cancer	bipolar disorder	bone cancer	brain cancer
breast cancer	celiac disease	cervical cancer	chronic kidney failure	chronic obstructive pulmonary disease	colon cancer

Exécution de l'algorithme RW

- En temps optimisé X10
- Améliorations documentées

Avant

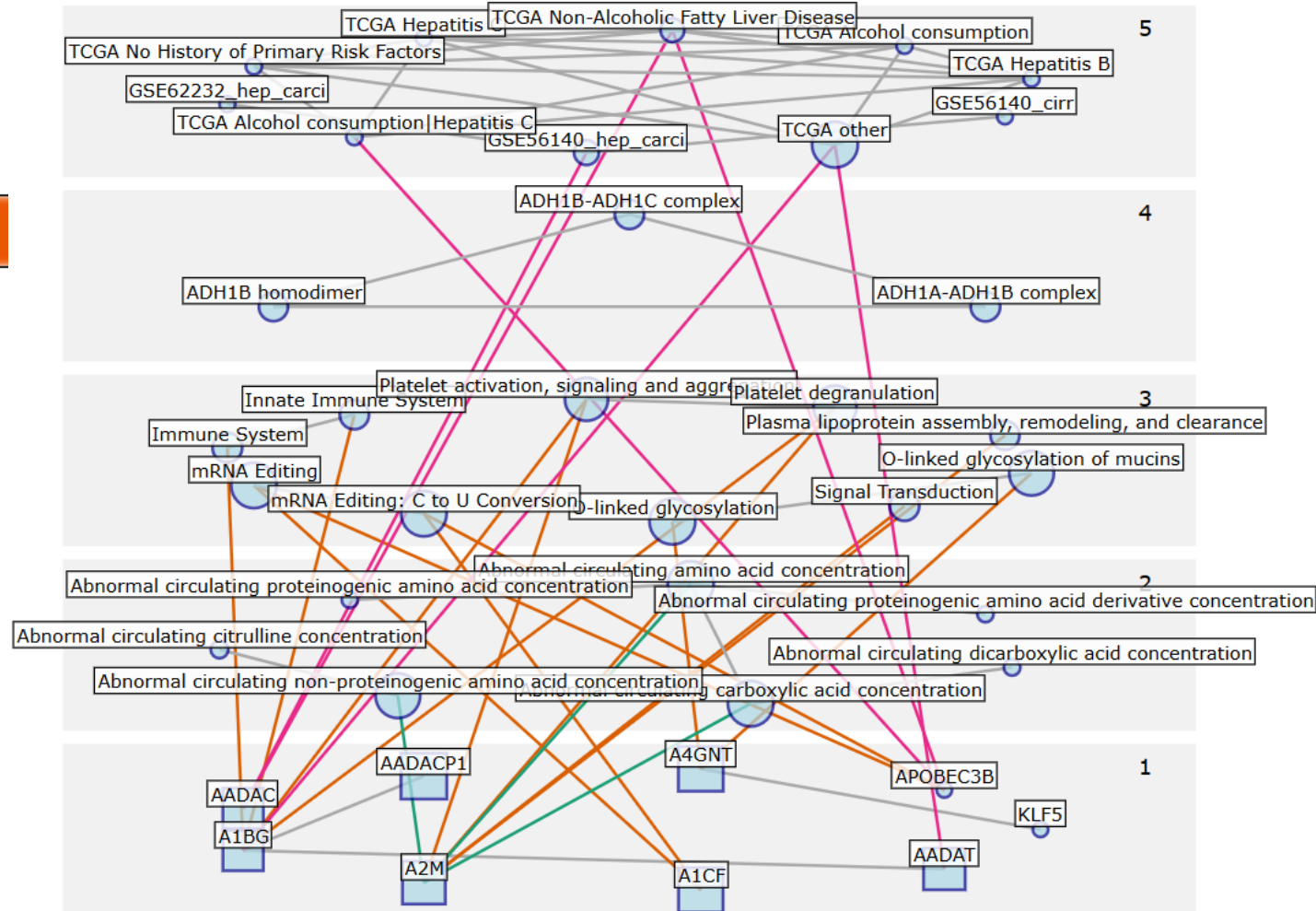


Après

Affichage du graphe

- Interactif
- Lisible
- Peut être exporté

Interactive Multiplex Graph



Types of interaction

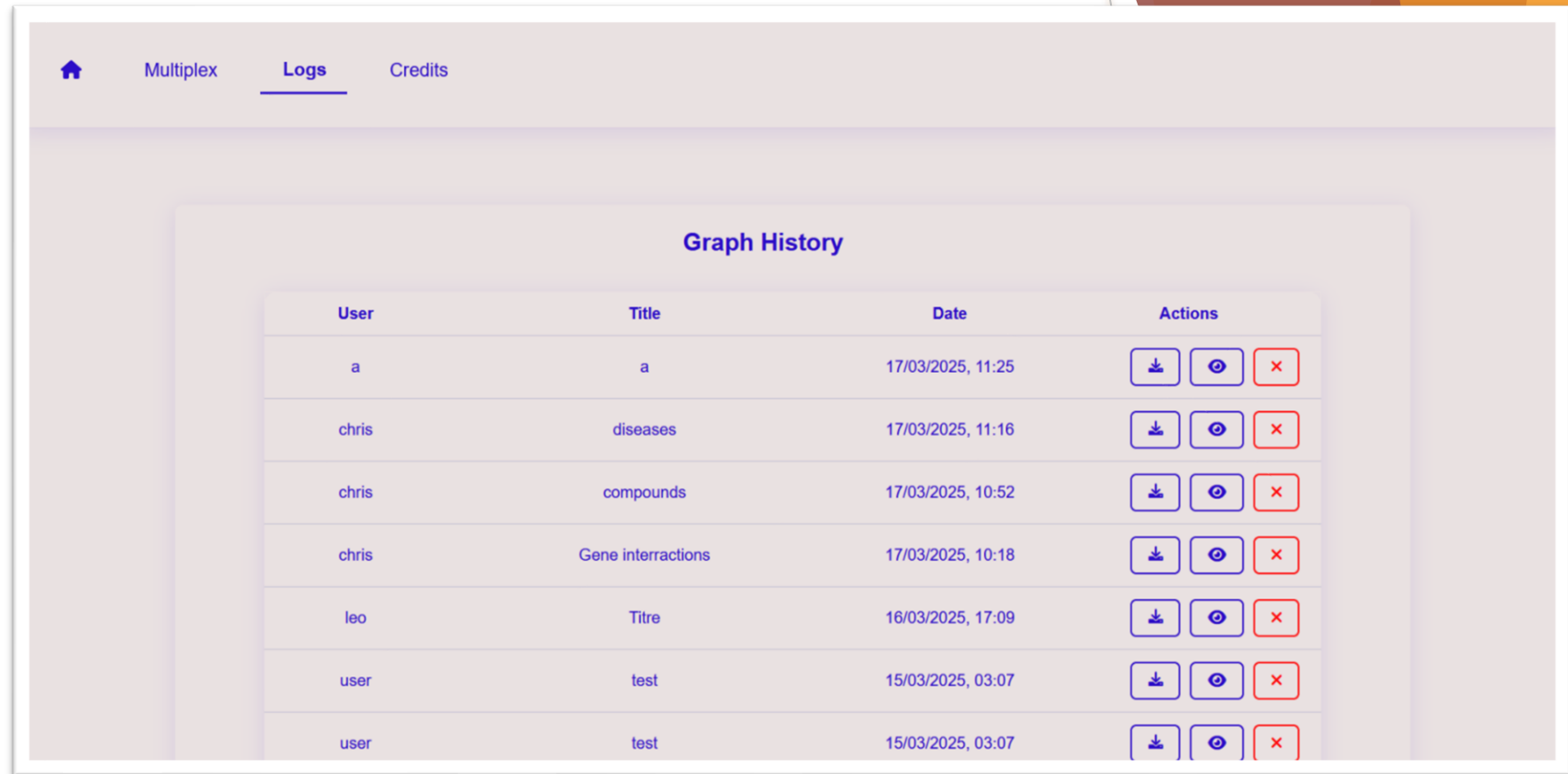
- Intra-layer interaction - 1
- Inter-layer interaction - 1 and 3
- Inter-layer interaction - 1 and 5
- Intra-layer interaction - 2
- Inter-layer interaction - 1 and 2
- Intra-layer interaction - 3
- Intra-layer interaction - 4
- Intra-layer interaction - 5
- Seeds

Parameters

- Iterations : 1
- Number of nodes per layer : 10
- Seeds : A1BG, A1CF, A2M, A4GNT, AACS, AADAC, AADACP1, AADAT
- Restart (%) : 0.7
- Missing seeds : AACS

Sauvegarde en BDD

- Entrée et sortie
- ZIP téléchargeable



Graph History			
User	Title	Date	Actions
a	a	17/03/2025, 11:25	  
chris	diseases	17/03/2025, 11:16	  
chris	compounds	17/03/2025, 10:52	  
chris	Gene interactions	17/03/2025, 10:18	  
leo	Titre	16/03/2025, 17:09	  
user	test	15/03/2025, 03:07	  
user	test	15/03/2025, 03:07	  

Outils utilisés

Base de données:

- SQLite
- Stocke les informations sur les historiques de graphes

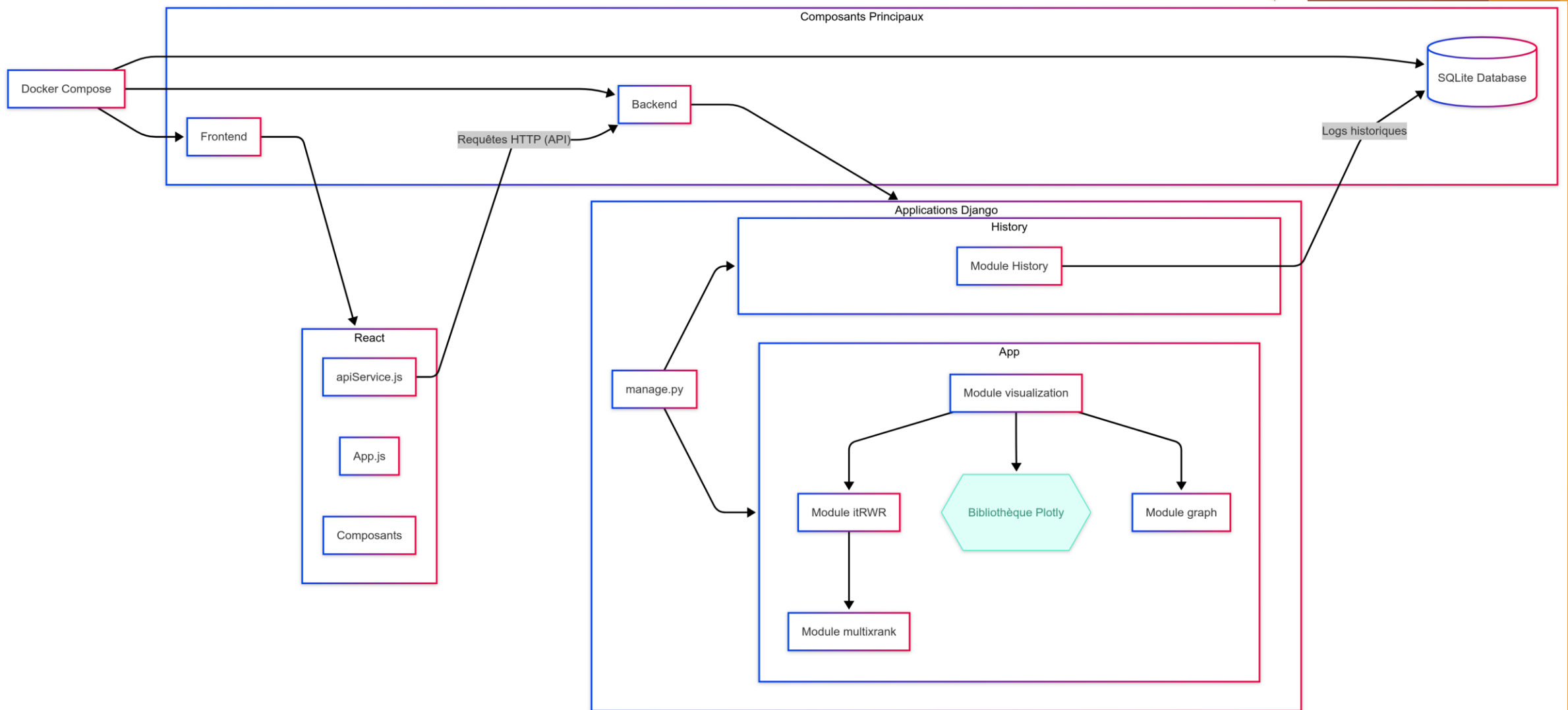
Backend:

- Django (Python)
- Gère la logique de traitement

Frontend:

- React
- Interface utilisateur → configurer et lancer les marches aléatoires, afficher les sous-graphes obtenus et consulter l'historique

Architecture



Approche générale et organisation des tests

Pourquoi tester?

- Garantir la qualité, prévenir les bugs, valider les fonctionnalités.
- Domaine de d'application: assurer le bon fonctionnement des structures complexes (graphes multiplexes)

Quand tester ?

- A la fin de chaque sprint et lors du développement avant la validation d'une user story (DOD)

Outils utilisés :

- unittest pour le backend,
- Jest et Selenium pour le frontend.

Les types de tests réalisés :

- Tests unitaires
- Tests d'intégration

Exemple de test d'intégration

```
1 import unittest
2 from unittest.mock import mock_open, patch
3 from ..itRWR.file_utils import write_config_file, read_seeds
4
5 class TestFileUtils(unittest.TestCase):
6     @patch("builtins.open", new_callable=mock_open, read_data="old_seed_path\nother_config\n")
7     @patch("os.path.join", side_effect=lambda *args: "/".join(args))
8     def test_write_config_file(self, mock_file):
9         """
10         Test the 'write_config_file' function to ensure it correctly writes the updated configuration file.
11
12         - Mocks the 'open' function to simulate file reading and writing.
13         - Mocks 'os.path.join' to simulate path joining behavior.
14         - Verifies that the first line of the output file is replaced with the new seed file path.
15         - Verifies that the remaining lines of the input file are written unchanged.
16         """
17         # Call the function to write the configuration file
18         write_config_file("config.txt", "output_folder", 123)
19
20         # Assert that the new seed file path is written as the first line
21         mock_file().write.assert_any_call("seed: output_folder/seeds_123.txt\n")
22
23         # Assert that the other configuration lines are written unchanged
24         mock_file().write.assert_any_call("other_config\n")
```

Exemple de test unitaire

```
7 # Create a factory for generating mock API requests
8 factory = APIRequestFactory()
9
10 class TestViews(unittest.TestCase):
11
12     @patch('app.views.list_multiplex_networks')
13     def test_multiplex_list(self, mock_list):
14         """
15         Test the 'multiplex_list' view.
16
17         - Mocks the 'list_multiplex_networks' function to simulate the response.
18         - Verifies that the view returns a 200 status code.
19         - Verifies that the response contains the expected list of multiplex networks.
20         """
21         mock_list.return_value = ['multiplex1', 'multiplex2']
22         request = factory.get('/fake-url/')
23         response = multiplex_list(request)
24         self.assertEqual(response.status_code, 200)
25         self.assertEqual(json.loads(response.content), {'multiplexes': ['multiplex1', 'multiplex2']})
```


File Edit Selection View Go Run Terminal Help

graphes-multiplex

EXPLORER

- GRAPHES-MULTIPLEX
 - backend
 - app
 - __pycache__
 - graph
 - itRWR
 - migrations
 - multixrank
 - tests
 - _init_.py
 - selenium_layer_selection.py
 - selenium_log.py
 - selenium_select_seed.py 9+, M
 - selenium_upload_zip.py
 - test_create_arbo_from_zip.py
 - test_file_utils.py
 - test_generate_config.py
 - test_get_layer_mapping.py
 - test_get_layer_names.py
 - test_get_zip_file.py
 - test_list_layers.py
 - test_list_unique_nodes.py
 - test_ranking_utils.py
 - test_sif_utils.py
 - test_views.py
 - test_write_config_file.py
 - test2.zip
 - utils
 - visualization
 - _init_.py
 - admin.py
 - apps.py
 - itRWR_controller.py
 - OUTLINE
 - TIMELINE

JS MultiplexMA.js 3, M

selenium_select_seed.py 9+, M

JS MultiplexMA.js (Working Tree) 3, M

views.py (Working Tree)

views.py

JS Home.js

```
backend > app > tests > selenium_select_seed.py > TestSelectSeed > test_fullexec
1 import unittest
2 import time
3 import os
4 import pytest
5 import json
6 from selenium import webdriver
7 from selenium.webdriver.common.by import By
8 from selenium.webdriver.support.ui import WebDriverWait, Select
9 from selenium.webdriver.support import expected_conditions as EC
10 from selenium.webdriver.common.action_chains import ActionChains
11 from selenium.webdriver.support.wait import WebDriverWait
12 from selenium.webdriver.common.keys import Keys
13 from selenium.webdriver.common.desired_capabilities import DesiredCapabilities
14 from selenium.common.exceptions import TimeoutException
15
16 class TestSelectSeed(unittest.TestCase):
17     @classmethod
18     def setUpClass(cls):
19         # Configure browser options
20         cls.options = webdriver.ChromeOptions()
21         # cls.options.add_argument("--headless") # Remove for visible browser
22         cls.driver = webdriver.Chrome(options=cls.options)
23         cls.wait = WebDriverWait(cls.driver, 15)
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
```

PROBLEMS 13 COMMENTS OUTPUT DEBUG CONSOLE TERMINAL PORTS

OK

(venv) PS C:\Users\chris\Desktop\graphes-multiplex\backend\app\tests> python .\selenium_select_seed.py -v

DevTools listening on ws://127.0.0.1:55701/devtools/browser/ca274175-cae3-4419-9381-aa817668330e

test_fullexec (__main__.TestSelectSeed.test_fullexec) ... Created TensorFlow Lite XNNPACK delegate for CPU.

ok

Ran 1 test in 21.218s

OK

(venv) PS C:\Users\chris\Desktop\graphes-multiplex\backend\app\tests> python .\selenium_select_seed.py -v

python tests

powershell

main* 0 13 Git Graph

Not Committed Yet Ln 99, Col 26 Spaces: 4 UTF-8 CRLF {} Python 3.12.9 64-bit (Microsoft Store) Go Live

CI

- ▶ Tests automatiques aux commits
- ▶ Problème GitLab-runner docker in docker au CREMI -> installation d'un GitLab-runner sur une de nos machines

 Passed ⌚ 00:03:03 📅 1 day ago	changement sur l'utilisation sans d... #26551  main  8fffe56a 		
 Passed ⌚ 00:02:54 📅 1 day ago	Merge branch 'main' into selenium #26549  main  3c050caf 		 → 

Déploiement & production

- ▶ Une version production (nouveau fichier Docker, paramètres modifiés, gestion des clés)
- ▶ Build front servi par un module du serveur django sur gunicorn
- ▶ Un script de déploiement (choix du port et du nom d'hôte)

Résumé des sprints

Sprint 1

- Construction
 - Back
 - Front + Maquette
 - Graphes
- Apprivoisement de l'algo
- Résolution des pbs principaux

Sprint 2

- Optimisation
- Intégration+API
- Bcp de tests
- Avancées Front
- Implémentation des retours (UI, graphe)

Sprint 3

- Finalisation Liaison Front/Back
- Démo
- Fixes
- Refactoring
- Doc

Diagramme de flux cumulé

✓ A faire ✓ En cours ✓ A Tester ✓ Terminé

